Forward and Inverse Kinematics Analysis on Franka Emika Panda

Marlon Steve Rivas Morataya
Northeastern University
Robotics Mechanics and Controls
Boston, MA

***Abstract*—This paper examines and analyzes the 7-DOF Franka Emika Panda using forward and inverse kinematics. The objective of the analysis is to reconstruct a robot trajectory using MuJoCo as a simulation and calculation engine. The forward kinematics analysis is performed using the Product of Exponentials (POE) method from the book Modern Robotics [1] and is thoroughly compared with MuJoCo's forward kinematics to verify the analytical method. Then, a trajectory is defined for the robot to follow, given as end-effector poses, and a Newton-Raphson inverse kinematics method is used to obtain the desired joint positions and analyze how the method performs throughout the trajectory.**

***Index Terms*—Degrees of Freedom (DOF), Product of Exponentials (POE)**

I. Introduction

Kinematics in robotics is a fundamental concept that analyzes the motion of a robot, focusing on how the position, velocity, and acceleration of different robot components (joints, arms, end-effector) evolve as they move through the workspace. Kinematics is essential for designing and controlling robotic systems. There are two main types of kinematics: forward and inverse kinematics. Forward kinematics involves calculating the pose (rotation and translation) of the end-effector given a set of joint variables. Inverse kinematics involves determining the required joint variables to achieve a desired end-effector pose. This paper presents an analysis and comparison of analytical forward kinematics using the Product of Exponentials (POE) method, as introduced in *Modern Robotics*, and validates the results using MuJoCo. A trajectory is then defined as a sequence of end-effector poses, and a Newton-Raphson inverse kinematics solver is used to follow this trajectory.The performance of the inverse kinematics method is evaluated across a range of iterations, and improvements to the basic Newton-Raphson method are introduced to enhance stability and accuracy.

II. Analytical Forward Kinematics

The robot selected for this analysis is the 7-DOF Franka Emika Panda, which is used in assembly lines, material handling, mobile manipulation, and other industries. The MuJoCo model for the Franka Panda was obtained from the Google DeepMind MuJoCo Menagerie repository on GitHub [5].

The first step in calculating the analytical forward kinematics is to compute the M matrix from the home configuration, expressed as a homogeneous transformation matrix in $SE(3)$ in the form of:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \tag{1}$$

- R - is an $SO(3)$, 3×3 rotation matrix
- p - is the position vector (3×1)

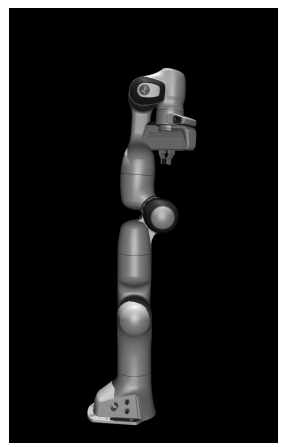


Fig. 1: Franka Emika Panda Home Configuration

Using Fig. 1 as a reference, and by setting the robot to the home configuration, it is possible to obtain the required end-effector rotation matrix and position vector from MuJoCo. The following pipeline outlines the necessary steps to compute the forward kinematics analytically:

$$\underbrace{S_i = (\omega_i, v_i)}_{\text{Screw axis}} \rightarrow \underbrace{[S_i]\theta_i = \begin{bmatrix} [\omega_i] & v_i \\ 0 & 0 \end{bmatrix} \theta_i}_{\text{Twist (se(3))}} \rightarrow \underbrace{T = e^{[S_i]\theta_i}}_{\text{Matrix exponential}} \tag{2}$$

The first step of the process is to obtain the screw axis $S_i = (\omega_i, v_i)$ for each joint, where ω_i is the unit rotation axis expressed in the world frame and v_i is the linear velocity component of the twist.

The next step is to convert the screw axis into a skew-symmetric matrix representation, which can then be exponentiated using Rodrigues' rotation formula:

$$R = e^{[\hat{\omega}]\theta} = I + \sin\theta[\hat{\omega}] + (1 - \cos\theta)[\hat{\omega}]^2 \tag{3}$$

where:

- $\hat{\omega}$ - is the unit rotation axis
- θ - is the joint angle

The matrix exponential in $SE(3)$ can be constructed by combining Eq. 3 for the rotational component and using the $G(\theta)$ matrix multiplied by v_i for the translational component, as shown below:

$$T = e^{[S]\theta} = \begin{bmatrix} e^{[\omega]\theta} & G(\theta)v \\ 0 & 1 \end{bmatrix} \tag{4}$$

where:

$$G(\theta)v = \left(I\theta + (1-\cos\theta)[\omega] + (\theta - \sin\theta)[\omega]^2\right) v \tag{5}$$

Following the process in Eq. 2, it is possible to map from $[S]\theta \in se(3)$ to $T \in SE(3)$, resulting in a homogeneous transformation matrix.

$$T = e^{[S_1]\theta_1} \cdots e^{[S_n]\theta_n} M \tag{6}$$

Using Eq. 6, known as the Product of Exponentials formulation, the forward kinematics can be obtained by multiplying the matrix exponentials of each joint to compute the end-effector pose.

Following the kinematic pipeline, it was necessary to obtain the screw axes of the robot. From the robot model, it can be observed that all joints (0–7) rotate about their respective local z-axes, giving an initial angular velocity vector $\omega = (0, 0, 1)$ in their local frames.

Using the robot model and MuJoCo, it was possible to obtain the world position of each joint. Since this information is provided in the local (body) frame, it must be transformed into the world frame using: $\omega_s = R_{sb}\omega_b$. To compute the linear component of the screw axis, the following was used:

$$v = \omega \times -q \tag{7}$$

- ω - is a 3×1 angular velocity vector
- q - is a 3×1 position vector representing a point on the joint axis

Using the joint positions obtained from the robot model and MuJoCo, the screw axis for each joint was computed. The analytical forward kinematics were then implemented using the *Modern Robotics* library in Python [2], where the function `FKinSpace` executes the pipeline described in Eq. 2 and Eq. 6, given the screw axes, joint angles θ_i, and the home configuration matrix M, following the formulation presented in [1]. To validate the POE formulation, 100 random joint configurations were generated. The position error was computed using the Euclidean norm, while the rotational error was obtained using the geodesic distance between rotation matrices. The results, shown in Table I, indicate negligible error between the analytical and MuJoCo forward kinematics, confirming the correctness of the implementation. The validated analytical forward kinematics are later used as a reference in the inverse kinematics section to evaluate the results obtained.

Metric	Position Error (m)	Rotation Error (rad)
Minimum	6.94e-17	3.65e-08
Mean	6.40e-16	7.23e-08
Maximum	1.56e-15	8.94e-08

TABLE I: Forward Kinematics over 100 Configurations

III. Newton-Raphson Inverse Kinematics

To construct the inverse kinematics solver, the twist vector $\mathcal{V} \in \mathbb{R}^6$, consisting of rotational and linear components (ω, v), is required. Following the pipeline depicted in Eq. 8, it is possible to obtain the twist vector from a transformation matrix.

$$\underbrace{T \in SE(3)}_{\text{Transformation matrix}} \rightarrow \underbrace{[S]\theta = \log(T) \in se(3)}_{\text{Matrix logarithm}} \rightarrow \underbrace{\mathcal{V} = (S\theta)^\vee}_{\text{Twist vector}} \tag{8}$$

Using MuJoCo, it is possible to obtain both the rotational and translational Jacobians of the end-effector. These components are combined into a single Jacobian $J \in \mathbb{R}^{6\times n}$, as shown in Eq. 9:

$$J_s(q) = \begin{bmatrix} J_{rot}(q) \\ J_{pos}(q) \end{bmatrix} \in \mathbb{R}^{6\times n} \tag{9}$$

where:

- J_{rot} - maps joint velocities to angular velocity of the end-effector
- J_{pos} - maps joint velocities to linear velocity of the end - effector

Since the Jacobian obtained from MuJoCo is expressed in the space (world) frame, it is necessary to convert it to body (local) coordinates. This is done using the adjoint transformation of the inverse end-effector pose, as shown in Eq. 10:

$$J_b(q) = \left[Ad_{T_{sb}^{-1}}\right] J_s(q) \tag{10}$$

The Newton-Raphson method is an iterative technique used to find the roots of nonlinear equations. This approach can be adapted to inverse kinematics to compute an approximate solution for the joint configuration that achieves a desired end-effector pose.To apply the Newton-Raphson method to inverse kinematics, an initial guess of the joint configuration and a desired end-effector pose are required. At each iteration, the error between the desired pose and the current pose is computed. The joint configuration is then updated using the pseudo-inverse of the Jacobian, as shown in Eq. 11:

$$\Delta\theta = J^\dagger(\theta^k)\left(x_d - f(\theta^k)\right) \tag{11}$$

Making use of the Newton-Raphson method, the pseudoinverse of the body Jacobian is computed and used to update the joint configuration iteratively. The change in joint configuration is obtained by left-multiplying the twist error $\mathcal{V}_{err}$ by the pseudoinverse of the body Jacobian, resulting in the update term Δq. This update is then added to the

current joint estimate to obtain a new configuration closer to the desired pose.To improve stability, a scaling factor α is introduced to reduce the step size, as large updates may lead to instability near singular configurations. The update rule is given by:

$$q_{k+1} = q_k + \alpha \left(J_b^{\dagger}(q_k)\, \mathcal{V}_{err} \right) \tag{12}$$

where:

- $J_b^{\dagger}(q_k)\, \mathcal{V}_{err}$ – is the joint update Δq
- α – is a scaling factor used to improve stability (selected as $\alpha = 0.1$)

Using the standard Newton inverse kinematics method, the solver performed well for trajectories that did not approach singular configurations. However, as the robot approached singularities, the pseudoinverse produced very large updates Δq, since the Jacobian becomes ill-conditioned and involves division by values close to zero. This caused the robot to move in an unstable and unpredictable manner, sometimes resulting in configurations that are not physically feasible.To address this issue, the Levenberg-Marquardt method, also known as Damped Least Squares (DLS), was implemented to improve stability near singular configurations. The DLS method introduces a damping term $\lambda^2 I$, which regularizes the inversion and prevents numerical instability. The updated expression for Δq is given by:

$$\Delta q = J_b^T \left(J_b J_b^T + \lambda^2 I \right)^{-1} \mathcal{V}_{err} \tag{13}$$

where:

- λ – Can be selected to balance convergence and numerical stability (sel: 0.05)
- I - Identity matrix for this instance (6x6)

Using this modified update improved the behavior of the robot in challenging regions of the trajectory, as it prevented large, unstable joint updates and resulted in smoother and more consistent motion when approaching difficult configurations.The implementation of the algorithm was set to run for a fixed number of iterations or until the pose error converged below a predefined threshold. The initial joint configuration q_{guess} was taken from the starting pose of the robot (typically the home configuration). At each iteration, a new set of joint positions was computed using the inverse kinematics update. The twist error between the desired and current pose was then evaluated to determine whether convergence had been achieved or if additional iterations were required.

IV. Trajectory Generation

The process of selecting and generating the trajectory was designed to guide the robot to a target location behind a cross-shaped obstacle. The trajectory was generated as a sequence of desired end-effector poses in $SE(3)$. Each segment was defined by specifying a start and end transformation, and intermediate poses were obtained through interpolation. The position component was interpolated linearly, while the orientation was smoothly varied using rotation matrices. Each segment was discretized into small increments, producing a sequence of target poses (T_{di}), which were then used as inputs to the inverse kinematics solver. The trajectory was divided into multiple segments, where each segment was defined to achieve a specific motion while avoiding collisions and ensuring stable inverse kinematics performance.Within the workspace, a cross-shaped obstacle was placed at a distance of 0.5 meters from the robot base to create a meaningful and constrained motion planning problem. The trajectory was divided as follows:

1) Segment 1 - Starting from the home configuration, the robot moves forward along the x-direction.
2) Segment 2 - The robot moves in a lateral direction (y and -z) to avoid the obstacle.
3) Segment 3 - The end-effector is rotated by a specified angle to further avoid contact with the obstacle.
4) Segment 4 - After rotating, the robot moves forward along the x-direction to approach the goal region.
5) Segment 5 - The end-effector moves diagonally toward the center of the cross while maintaining clearance from the obstacle.
6) Segment 6 - The end-effector moves diagonally toward the final goal pose while adjusting its position to safely reach the target.

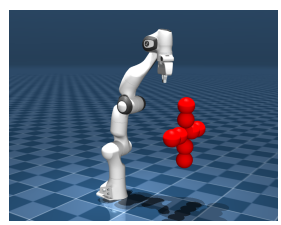


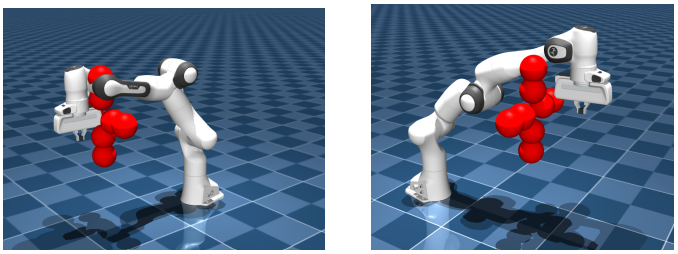
Fig. 2: Franka Panda with Cross in Workspace

As shown in Fig. 2, the robot follows the defined trajectory while navigating around obstacles in the workspace. The trajectory was selected because it combines simple motions that result in a complex and meaningful path. Due to the placement of the objects, the robot must maneuver around the obstacle to avoid direct contact while also reaching a target located behind it. This trajectory encourages the Franka Panda to utilize most of its joints, including coordinated motion and end-effector rotation, in order to achieve the desired goal pose.

V. Trajectory Tracking Results

In the initial tests using the standard Newton inverse kinematics method, it was observed that the robot exhibited unstable behavior while following the trajectory. This resulted in inconsistent motion and the generation of undesired intermediate poses, which in a real-world scenario could potentially damage the robot. As shown in Fig. 3a, the robot is able to reach the desired target pose; however, it does so by passing through obstacles and exhibiting unrealistic joint configurations.

After introducing the Damped Least Squares (DLS) method, the trajectory tracking behavior became significantly smoother,



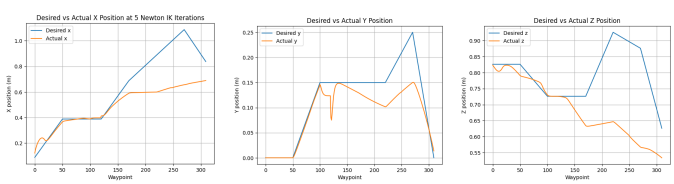
(a) Without DLS (b) With DLS

Fig. 3: Comparison of final robot pose with and without Damped Least Squares (DLS).

and no undesired configurations were observed when approaching singular configurations. The total number of poses generated for the selected trajectory was 310. The inverse kinematics solver was initially configured to run for 5 iterations per waypoint. Additional tests were performed by increasing the number of iterations, as summarized in Table II. From the results, it can be observed that the translational error remains relatively consistent across different iteration counts, with a slight increase as the number of iterations grows. Similarly, the rotational error remains within a consistent range, with the best overall performance observed around 50 iterations.It was also observed that enforcing joint limits through clipping increases the overall error. As shown in Table II, the case with 5 iterations without clipping produces slightly lower translational and rotational errors. However, to maintain physical realism and ensure safe joint configurations, joint limit clipping was retained in the final implementation. Although the numerical results suggest that the error slightly increases with a higher number of IK iterations, the visual behavior of the robot improves. With a low number of iterations (e.g., 5), the robot exhibits oscillatory or "wiggly" motion in certain segments of the trajectory. This behavior is significantly reduced as the number of iterations increases, resulting in smoother and more stable motion overall.

	Translational Error (m)				Rotational Error (rad)			
Iterations	RMSE	Mean	Std	Max	RMSE	Mean	Std	Max
5 (clip)	0.2582	0.1832	0.1820	0.5399	0.0536	0.0405	0.0351	0.2578
5 (no clip)	0.2576	0.1794	0.1848	0.5399	0.0497	0.0357	0.0346	0.2578
50	0.2607	0.1833	0.1854	0.5400	0.0600	0.0388	0.0458	0.2460
100	0.2624	0.1848	0.1864	0.5401	0.0671	0.0403	0.0536	0.3283

TABLE II: Trajectory Error Metrics for Different Iterations



(a) p_x (b) p_y (c) p_z

Fig. 4: Comparison of desired and actual end-effector positions along each axis

Another metric evaluated was the error between the desired and actual end-effector position along each individual axis. As shown in Fig. 4, all three axes closely follow the desired trajectory at the beginning. However, as the robot progresses through the more complex segments of the trajectory, deviations from the desired path become more noticeable. This behavior is particularly evident in the middle portions of the trajectory, where changes in direction and orientation increase the difficulty of the inverse kinematics solution. Despite these deviations, the end-effector is still able to reach the final pose with reasonable accuracy.

The final pose error was measured as:

1) Translational error magnitude = 0.176 m
2) Rotational error = 0.0332 rad

These results are consistent with the values reported in Table II, as well as the trends observed in Fig. 4. Overall, the inverse kinematics solver demonstrates acceptable performance for trajectory tracking, although there is room for improvement. The increase in error and plateau behavior observed in certain regions of the trajectory suggest limitations due to factors such as joint constraints, limited iteration count, and the complexity of the motion. Further improvements could be explored by refining the trajectory design, adjusting solver parameters, or testing alternative trajectories that reach the same goal from different approach directions. This would allow for a comparison of how trajectory selection influences tracking accuracy. The Newton-based inverse kinematics method proved to be an effective and computationally efficient approach for solving a complex nonlinear problem such as inverse kinematics, especially when combined with techniques such as damping to improve stability.

Conclusion

In conclusion, this project provided valuable insight into how forward and inverse kinematics can be used to analyze and control a robotic system within its workspace, and how these concepts relate to the material covered throughout the semester. Through the implementation and analysis of both forward and inverse kinematics, a deeper understanding of robot motion, trajectory tracking, and the challenges associated with singularities and numerical stability was developed.For future work, improvements can be made by developing more efficient inverse kinematics methods to reduce tracking error, as well as incorporating a control strategy to regulate the robot's motion more effectively. Additionally, designing and analyzing a smaller robotic system using forward and inverse kinematics would further reinforce the concepts explored in this project.The full code implementation is available on the GitHub repository [3], and a demonstration video is provided on YouTube [4], showcasing the robot successfully following and completing the trajectory.

VI. References

References

[1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, U.K.: Cambridge University Press, 2017.

[2] K. M. Lynch and F. C. Park, "Modern Robotics Code Library," GitHub repository. [Online]. Available: https://github.com/NxRLab/ModernRobotics. [Accessed: Apr. 22, 2026].
[3] M. Steve Rivas, "Newton Raphson IK and FK – source code." [Online]. Available: https://github.com/marlonsrivas-NU/Newton-Raphson-IK-and-FK. [Accessed: Apr. 22, 2026].
[4] M.Steve Rivas, "Damped Least Squares Newton-Raphson IK" [Online]. Available: https://youtu.be/hAesNi5Djbc. [Accessed: Apr. 22, 2026].
[5] MuJoCo Menagerie Contributors, *MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo*, Google DeepMind, 2022. Available at: https://github.com/google-deepmind/mujoco_menagerie, commit `1b86ece`.
[6] Wang, X., Li, Y., and Zhao, J., "Enhanced Gaussian damped least squares method for inverse kinematics of redundant robots," *IEEE Access*, vol. 8, pp. 123456–123467, 2020.

Appendix

Appendix A: Python Code

Forward Kinematics — Franka Panda

ME5250 Robot Mechanics & Controls | Project 2

This notebook implements Forward Kinematics (FK) using the Product of Exponentials (PoE) and Compares it with MuJoCo's Forward Kinematics

In [ ]:
```
import mujoco
import mujoco.viewer
import numpy as np
from matplotlib import pyplot as plt
import modern_robotics as mr

np.set_printoptions(precision=4, suppress=True, linewidth=120)
DEBUG = 0

#Panda Joint Limits
q_min = np.array([
    -2.8973,   # joint1
    -1.7628,   # joint2
    -2.8973,   # joint3
    -3.0718,   # joint4
    -2.8973,   # joint5
    -0.0175,   # joint6
    -2.8973    # joint7
])

q_max = np.array([
     2.8973,   # joint1
     1.7628,   # joint2
     2.8973,   # joint3
    -0.0698,   # joint4
     2.8973,   # joint5
     3.7525,   # joint6
     2.8973    # joint7
])


#Load Mujoco model from XML
# MjModel contains the model description (geometry, joints, actuators, etc)
model = mujoco.MjModel.from_xml_path(r"C:\Users\marli\OneDrive\Escritorio\Grad_2026\RobMechControls\ME5250-RMC\project2\franka_

# MjData contains the simulation state (pos, vels, forces, etc)
data = mujoco.MjData(model)

# Set the simulation state to the default config
mujoco.mj_resetData(model,data)

print(f"Number of degrees of freedom (DOF): {model.nv}")
print(f"Number of actuators: {model.nu}")
print(f"Number of bodies: {model.nbody}")

mujoco.mj_forward(model,data)

print(f"Number of qpos: {model.nq}")
print(f"Number of joints: {model.njnt}")
print(f"Number of sites: {model.nsite}")
```

Obtain and compute Forward Kinematics

- M matrix at home configuration
- Screw Axis
- Use Modern Robotics Library to Finish POE FK method

In [ ]:
```
#Reset and set a specific configuration
mujoco.mj_resetData(model,data)

#Get EE site ID
site_id = model.site('gripper').id

#Compute forward kinematics
mujoco.mj_forward(model,data)

ee_pos = data.site_xpos[site_id]
ee_rot = data.site_xmat[site_id].reshape(3,3)


if DEBUG == 1:
    print(f"\tEE position: [{ee_pos[0]:.3f}, {ee_pos[1]:.3f}, {ee_pos[2]:.3f}]")
    print(f"EE Rotation: [{ee_rot}]")

def Get_Screw_axis(n):
    #Get body id for link
    link =f"link{n}"
    body_id = model.body(link).id
    #print(f"\nLink{n} id: {body_id}")

    #obtain world position for Link1
    L = data.xpos[body_id]
    #print(f"Link{n}= {L}")

    omega_local = [0,0,1]

    #Get the world orientation of Link1
    R = data.xmat[body_id].reshape(3,3)
    #print(f"\nRotation:\n {R}")
```

```
    #Rotate local frame to world coordinates
    omega_world = R @ omega_local
    #print(f"\n omega world = {omega_world}")
    #Compute Screw Axis
    v = np.cross(omega_world,-L)
    #print(f"Screw_axis{n} = {v}")

    Screw_axis = np.hstack((omega_world,v))
    print(f"Screw axis{n} = {Screw_axis}")

    return Screw_axis
def POE_FK(M,slist,thetalist):
    #obtain Screw Axis
    T = mr.FKinSpace(M,slist,thetalist)
    return T
def Pose_error(T_mj, T_fk):
    Rot_fk = T_fk[:3, :3]
    pos_fk = T_fk[:3, 3]

    Rot_mj = T_mj[:3, :3]
    pos_mj = T_mj[:3, 3]
    # Check for error:

    # Check Translational Error
    pos_error = np.linalg.norm(pos_fk - pos_mj)

    # Check Rotational Error if False then Rotation is not similar/ or similar enough.
    # True - means rotation passed and is close enough
    R_err = Rot_fk.T @ Rot_mj

    cos_theta = (np.trace(R_err) - 1.0) / 2.0
    cos_theta = np.clip(cos_theta, -1.0, 1.0)
    rot_error = np.arccos(cos_theta)

    return pos_error, rot_error
#Get M transform matrix from home position
M = np.eye(4)
M[:3,:3] = ee_rot
M[:3,3] = ee_pos
print(f"M matrix: \n {M}")

joints = model.njnt - 2
slist = []


for i in range(1,joints+1):
    screw = Get_Screw_axis(i)
    slist.append(screw)

slist = np.column_stack(slist)
#Home position Joint angles
theta_list = np.array((0.5,1,0.8,0.3,0.5,0.8,1))

#Using Modern Robotics Library
T_fk = POE_FK(M,slist,theta_list)

print(f"\nObtained POE T = \n{T_fk}")

#Use mujoco to check (everything up to the 7th link):
data.qpos[:7] = theta_list

#Get Mujoco EE pose
mujoco.mj_forward(model,data)

#Get end effector site ID (from xml file)
site_id = model.site("gripper").id

pos_mj = data.site_xpos[site_id].copy()
Rot_mj = data.site_xmat[site_id].reshape(3,3).copy()

T_mj = np.eye(4)

T_mj[:3,:3] = Rot_mj
T_mj[:3,3] = pos_mj

print(f"\nObtained mujoco T_mj = \n{T_mj}")
```

Test the Forward Kinematics POE method vs Mujoco Forward Kinematics

over a specific amount of configurations and compare its results

In [ ]:

```
# Test many configurations
total_pos_error = []
total_rot_error = []

if DEBUG == 1:
    for k in range(100):

        theta = np.random.uniform(low=q_min,high=q_max)
        theta_list = theta

        T_fk = POE_FK(M,slist,theta_list)

        #Use mujoco to check (everything up to the 7th link):
        data.qpos[:7] = theta_list
        data.qpos[7:] = 0.0

        #Get Mujoco EE pose
        mujoco.mj_forward(model,data)

        #Get end effector site ID (from xml file)
        site_id = model.site("gripper").id
```

```
    pos_mj = data.site_xpos[site_id].copy()
    Rot_mj = data.site_xmat[site_id].reshape(3,3).copy()

    T_mj = np.eye(4)

    T_mj[:3,:3] = Rot_mj
    T_mj[:3,3] = pos_mj

    #print(f"Mujoco pose \n {T_mj}")
    #print(f"\nPOE pose \n {T_fk}")

    pos_error, rot_error = Pose_error(T_mj,T_fk)
    print(f"\nTranslational Error is within: {pos_error}\n")
    print(f"Rotational Error is within: {rot_error}\n")
    total_pos_error.append(pos_error)
    total_rot_error.append(rot_error)

max_pos_error = np.max(total_pos_error)
min_pos_error = np.min(total_pos_error)
mean_pos_error = np.mean(total_pos_error)


max_rot_error = np.max(total_rot_error)
min_rot_error = np.min(total_rot_error)
mean_rot_error = np.mean(total_rot_error)

print("\nFK Validation Summary")
print(f"Position Error  | min: {min_pos_error:.6e} m, mean: {mean_pos_error:.6e} m, max: {max_pos_error:.6e} m")
print(f"Rotation Error  | min: {min_rot_error:.6e} rad, mean: {mean_rot_error:.6e} rad, max: {max_rot_error:.6e} rad")
```

Newton-Raphson Inverse Kinematics — Franka Panda

ME5250 Robot Mechanics & Controls | Project 2

This notebook implements Inverse Kinematics (IK) using a Damped Least Squares Newton-Raphson solver on the Franka Emika Panda robot in MuJoCo.

In [ ]:
```
import mujoco
import mujoco.viewer
import numpy as np
import mediapy as media
from matplotlib import pyplot as plt
import modern_robotics as mr

np.set_printoptions(precision=4, suppress=True, linewidth=120)
DEBUG = 0

#Load Mujoco model from XML
# MjModel contains the model description (geometry, joints, actuators, etc)
model = mujoco.MjModel.from_xml_path(r"franka_emika_panda\mjx_scene.xml")

# MjData contains the simulation state (pos, vels, forces, etc)
data = mujoco.MjData(model)

# Set the simulation state to the default config
mujoco.mj_resetData(model,data)

print(f"Number of degrees of freedom (DOF): {model.nv}")
print(f"Number of actuators: {model.nu}")
print(f"Number of bodies: {model.nbody}")

mujoco.mj_forward(model,data)

print(f"Number of qpos: {model.nq}")
print(f"Number of joints: {model.njnt}")
print(f"Number of sites: {model.nsite}")
```

---

1. Trajectory Generation

Defines helper functions to build a 7-segment end-effector trajectory using linear interpolation between waypoints.

In [ ]:
```
# Trajectory

def Make_pose(R, p):
    T = np.eye(4)
    T[:3, :3] = R
    T[:3, 3] = p
    return T


def Rotz(theta):
    return np.array([
        [np.cos(theta), -np.sin(theta), 0.0],
        [np.sin(theta),  np.cos(theta), 0.0],
        [0.0,            0.0,           1.0]
    ])


def linear_interpolation(pa, pb, s):
    # Linear interpolation between two points
    return (1 - s) * pa + s * pb


def Create_segment(p0, p1, R_st, n=50, end=0, theta_start=0.0, theta_end=0.0):
    segment = []

    # Convert once (not inside loop)
    theta_st = np.deg2rad(theta_start)
    theta_en = np.deg2rad(theta_end)

    # Endpoint flag
    endpoint_flag = bool(end)

    for s in np.linspace(0.0, 1.0, n, endpoint=endpoint_flag):

        # Linear position interpolation
        p = linear_interpolation(p0, p1, s)

        # Linear rotation interpolation about Z
        theta = linear_interpolation(theta_st, theta_en, s)
        R = R_st @ Rotz(theta)

        # Build pose
        T_traj = Make_pose(R, p)
        segment.append(T_traj)

    return segment


def Generate_trajectory(T_start,forward=0.30,sideways=0.15,downward=-0.1,final_forward=0.30,n=50):

    trajectory = []

    R0 = T_start[:3, :3]
    p0 = T_start[:3, 3]
```

```
    # Define key waypoints
    p1 = p0 + np.array([forward, 0.0, 0.0])
    p2 = p1 + np.array([0.0, sideways, downward])
    p3 = p2 + np.array([final_forward, 0.0, 0.0])
    p4 = p3 + np.array([0.2, 0.0, 0.2])
    p5 = p4 + np.array([0.2, 0.1, -0.05])
    p6 = p5 + np.array([-0.25, -0.25, -0.25])

    # Orientation change
    theta1 = np.deg2rad(-60)
    R1 = R0 @ Rotz(theta1)

    # Build trajectory

    # Segment 1 - Move forward
    trajectory += Create_segment(p0, p1, R0, n)

    # Segment 2 - Move sideways/downward
    trajectory += Create_segment(p1, p2, R0, n)

    # Segment 3 - Rotate end-effector in place
    trajectory += Create_segment(p2, p2, R0, n=20, theta_start=0.0, theta_end=-40.0)

    # Segment 4 - Move forward (with rotated orientation)
    trajectory += Create_segment(p2, p3, R1, n)

    # Segment 5 - Move upward
    trajectory += Create_segment(p3, p4, R1, n)

    # Segment 6 - Move to next point
    trajectory += Create_segment(p4, p5, R1, 50, 0)

    # Segment 7 - Move + rotate toward objective
    trajectory += Create_segment(p5, p6, R1, 40, 1, 0.0, 15.0)

    return trajectory
```

---

2. Inverse Kinematics - Damped Least Squares

Implements the Newton-Raphson IK update using the body Jacobian and Damped Least Squares (DLS) to handle singularities.

In [ ]:
```
#Inverting matrices in a better way than taking the inverse
def Mat_inv(T):

    Rot = T[:3,:3]
    p = T[:3,3]

    RotInv = Rot.T
    pinv = -RotInv @ p

    T_inv = np.eye(4)
    T_inv[:3,:3] = RotInv
    T_inv[:3,3] = pinv

    return T_inv

def Get_twist(T):

    se3mat = mr.MatrixLog6(T)

    #Twist
    V = mr.se3ToVec(se3mat)

    if DEBUG:
        print(V)

    return V

#Obtain Pose from q_guess using Mujoco and FK
def Get_pose(q_guess):

    #Get current pose from Mujoco
    data.qpos[:7] = q_guess
    data.qpos[7:] = 0.0
    mujoco.mj_forward(model,data)

    site_id = model.site("gripper").id

    p = data.site_xpos[site_id].copy()
    R = data.site_xmat[site_id].reshape(3,3).copy()

    T = np.eye(4)

    T[:3,:3] = R
    T[:3,3] = p

    return T

#Obtain the translational and rotational error
def Pose_error(T_goal, T_current):
    Rot_actual = T_current[:3, :3]
    pos_actual = T_current[:3, 3]

    Rot_goal = T_goal[:3, :3]
    pos_goal = T_goal[:3, 3]
    # Check for error:

    # Check Translational Error
    pos_error = np.linalg.norm(pos_actual - pos_goal)
```

```python
    # Check Rotational Error if False then Rotation is not similar/ or similar enough.
    # True - means rotation passed and is close enough
    R_err = Rot_actual.T @ Rot_goal

    cos_theta = (np.trace(R_err) - 1.0) / 2.0
    cos_theta = np.clip(cos_theta, -1.0, 1.0)
    rot_error = np.arccos(cos_theta)

    return pos_error, rot_error

#obtain Trajectory Error to check IK steps
def Traj_err(T_target,T_current):

    if DEBUG:
        print("T_current =\n", T_current)
        print("T_target =\n", T_target)

    T_curr_inv = Mat_inv(T_current)

    #T_err in body frame
    T_err = T_curr_inv @ T_target

    R_err = T_err[:3,:3]
    p_err = T_err[:3,3]

    p_err_mag = round(np.linalg.norm(p_err),3)
    if DEBUG:
        print(f"Translational error magnitude = {p_err_mag}")
        print(f"Translational error (body) = {p_err}")
        print(f"Rotational error = \n {R_err}")

    return T_err


#Convert Space Jacobian to Body Coordinates Jacobian
def Get_BodyJac(Jac_space,T_current):

    T_currentBody = Mat_inv(T_current)
    Jac_body = mr.Adjoint(T_currentBody)@Jac_space

    return Jac_body

#Inverse Kinematics Single Step
def IK_step(T_Target,q_guess,alpha=0.1):

    T_current = Get_pose(q_guess)
    #Calculate error based on current pose compared to our guess pose
    T_err = Traj_err(T_Target,T_current)
    #Compute 6-Vector Error twist (using MR library)
    V_err = Get_twist(T_err)

    site_id = model.site("gripper").id

    #Build Jacobians and obtain the EE space jacobian from Mujoco
    J_pos = np.zeros((3,model.nv))
    J_rot = np.zeros((3,model.nv))
    mujoco.mj_jacSite(model,data,J_pos,J_rot,site_id)

    #Combine Rotational and Position Jacobians to obtain right form use only 7 joints ignore last 2
    Jac_space = np.vstack((J_rot[:,:7],J_pos[:,:7]))

    #Get body Jacobian as EE position is being calculated from the body frame
    Jac_body = Get_BodyJac(Jac_space,T_current)


    #Use Damped Least Squares to solve issues with solver
    I = np.eye(6)
    lam = 0.05 #damping factor
    lam_sq = lam**2
    delta_q = Jac_body.T @ np.linalg.inv((Jac_body@Jac_body.T + lam_sq*I)) @ V_err

    ##Obtain pseudo-inverse jacobian
    #J_pinv = np.linalg.pinv(Jac_body)
    ##Obtain change in joints
    #delta_q = J_pinv @ V_err

    #Obtain new set of joints
    q_new = q_guess + alpha*delta_q

    # Panda joint limits
    q_min = np.array([-2.8973, -1.7628, -2.8973, -3.0718, -2.8973, -0.0175, -2.8973])
    q_max = np.array([ 2.8973,  1.7628,  2.8973, -0.0698,  2.8973,  3.7525,  2.8973])

    #q_new = np.clip(q_new,q_min,q_max)

    q_new = q_new

    return q_new

#Reset and set a specific config (home position)
mujoco.mj_resetData(model,data)
#Compute forward kinematics
mujoco.mj_forward(model,data)
site_id = model.site('gripper').id

q_start = data.qpos[:7].copy()
T_start = Get_pose(q_start)

Trajectory = Generate_trajectory(T_start)


#Get the current position of the joints from Mujoco
q_guess = q_start
q_path = []
```

```python
#Arrays for Error Calculation
px_actual = []
py_actual = []
pz_actual = []

px_desired = []
py_desired = []
pz_desired = []

#Obtain error
rotational_error = []
position_error = []

for T_target in Trajectory:

    error_norm = 0

    for i in range(5):

        #Perform Inverse Kinematics
        q_guess= IK_step(T_target,q_guess)

        #Get Pose from IK
        T_current = Get_pose(q_guess)

        #Calculate Error comparing Target and Current Pose
        T_err = Traj_err(T_target,T_current)
        V_err = Get_twist(T_err)

        error_norm = np.linalg.norm(V_err)

        if error_norm < 1e-3:
            break

        #print(f"iter {i} - The error norm is: {error_norm}")

    #Error Calculations
    p_target = T_target[:3,3]
    p_current = T_current[:3,3]

    px_actual.append(p_current[0])
    py_actual.append(p_current[1])
    pz_actual.append(p_current[2])

    px_desired.append(p_target[0])
    py_desired.append(p_target[1])
    pz_desired.append(p_target[2])

    print("\n")
    q_final = q_guess.copy()
    q_path.append(q_final)
    Trans_error, Rot_error = Pose_error(T_target,T_current)
    rotational_error.append(Rot_error)
    position_error.append(Trans_error)
```

Error Calculations after the loop

Calculate Error for Rotational and Translational Error for the whole trajectory and comparison between Goal pose and Actual End pose

In [ ]:
```python
# Obtain Error Metric Calculations over whole trajectory
Trans_error_max  = np.max(position_error)
Trans_error_mean = np.mean(position_error)
Trans_error_std  = np.std(position_error)
Trans_error_rmse = np.sqrt(np.mean(np.array(position_error)**2))

Rot_error_max    = np.max(rotational_error)
Rot_error_mean   = np.mean(rotational_error)
Rot_error_std    = np.std(rotational_error)
Rot_error_rmse   = np.sqrt(np.mean(np.array(rotational_error)**2))

print("=" * 50)
print(f"{'Trajectory Error Summary':^50}")
print("=" * 50)
print(f"{'Metric':<15} {'Translational (m)':>17} {'Rotational (rad)':>16}")
print("-" * 50)
print(f"{'RMSE':<15} {Trans_error_rmse:>17.6f} {Rot_error_rmse:>16.6f}")
print(f"{'Mean':<15} {Trans_error_mean:>17.6f} {Rot_error_mean:>16.6f}")
print(f"{'Std Dev':<15} {Trans_error_std:>17.6f} {Rot_error_std:>16.6f}")
print(f"{'Maximum':<15} {Trans_error_max:>17.6f} {Rot_error_max:>16.6f}")
print("=" * 50)



#Obtain Current Pos from Mujoco
T_actual = Get_pose(q_path[-1])
T_target = Trajectory[-1].copy()

#Calculate Error comparing Target and Current Pose
T_err_final = Traj_err(T_target,T_actual)
V_err_final = Get_twist(T_err_final)

Trans_error, Rot_error = Pose_error(T_target,T_actual)

print(f"\nTranslation error = {position_error}\n")
print(f"Rotational error = {Rot_error}")
```

Open MuJoCo and Start playing the Trajectory including the Cross Spheres

In [ ]:
```python
import time
import numpy as np
```

```python
import mujoco
import mujoco.viewer

def generate_cross_spheres(center, spacing=0.08, radius=2):
    positions = []
    x, y, z = center

    # Y axis
    for i in range(1, radius + 1):
        positions.append(np.array([x, y + i * spacing, z]))
        positions.append(np.array([x, y - i * spacing, z]))

    # Z axis
    for i in range(1, radius + 1):
        positions.append(np.array([x, y, z + i * spacing]))
        positions.append(np.array([x, y, z - i * spacing]))

    return positions


def play_q_path_with_markers(model, data, q_path, ball_positions,
                             pause=0.03, interp_steps=15, hold_final=True):
    with mujoco.viewer.launch_passive(model, data) as viewer:

        # Step through trajectory
        for i in range(len(q_path) - 1):
            q0 = np.array(q_path[i])
            q1 = np.array(q_path[i + 1])

            for a in np.linspace(0, 1, interp_steps, endpoint=False):
                if not viewer.is_running():
                    return

                q = (1 - a) * q0 + a * q1

                # Update robot pose
                data.qpos[:7] = q
                data.qpos[7:] = 0.0
                mujoco.mj_forward(model, data)

                # Clear user geoms
                viewer.user_scn.ngeom = 0

                # Add sphere markers
                for p in ball_positions:
                    mujoco.mjv_initGeom(
                        viewer.user_scn.geoms[viewer.user_scn.ngeom],
                        type=mujoco.mjtGeom.mjGEOM_SPHERE,
                        size=[0.05, 0, 0],   # radius = 5 cm
                        pos=p,
                        mat=np.eye(3).flatten(),
                        rgba=[1, 0, 0, 1]
                    )
                    viewer.user_scn.ngeom += 1

                viewer.sync()
                time.sleep(pause)

        # Hold final pose with markers visible
        if hold_final:
            data.qpos[:7] = q_path[-1]
            data.qpos[7:] = 0.0
            mujoco.mj_forward(model, data)

            while viewer.is_running():
                viewer.user_scn.ngeom = 0

                for p in ball_positions:
                    mujoco.mjv_initGeom(
                        viewer.user_scn.geoms[viewer.user_scn.ngeom],
                        type=mujoco.mjtGeom.mjGEOM_SPHERE,
                        size=[0.05, 0, 0],
                        pos=p,
                        mat=np.eye(3).flatten(),
                        rgba=[1, 0, 0, 1]
                    )
                    viewer.user_scn.ngeom += 1

                viewer.sync()
                time.sleep(0.01)

center_sphere = np.array([0.5,0,0.5])
ball_positions = generate_cross_spheres(center_sphere)

play_q_path_with_markers(model, data, q_path,ball_positions, pause=0.03, interp_steps=15)
```

Generate a Video of the Robot Following the trajectory

In [ ]:

```python
# Start at "home" config

data.qpos = np.append(q_final,[0.0,0.0])

#Create renderer for visualization
renderer= mujoco.Renderer(model, height=480, width=640)

#Get the camera ID
camera_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_CAMERA,"custom")

frames = []

print(f"\n Rendering Trajectory")

repeat_pose = 10
```

```
for q in q_path:

    data.qpos = np.append(q,[0.0,0.0])

    #update FK
    mujoco.mj_forward(model,data)

    #Update renderer with initial state and capture first frame
    renderer.update_scene(data, camera=camera_id)
    frame = renderer.render()

    for i in range(repeat_pose):
        frames.append(frame)

#Display the video
print(f"Simulation complete! Displaying Video")
media.show_video(frames, fps=30)
renderer.close()
```

Generate Error Plots for Translational Error

In [ ]:
```
waypoints = np.arange(len(px_desired))

plt.figure()
plt.plot(waypoints, px_desired, label="Desired x")
plt.plot(waypoints, px_actual, label="Actual x")
plt.xlabel("Waypoint")
plt.ylabel("X position (m)")
plt.title("Desired vs Actual X Position at 5 Newton IK Iterations")
plt.legend()
plt.grid(True)
plt.show()

plt.figure()
plt.plot(waypoints, py_desired, label="Desired y")
plt.plot(waypoints, py_actual, label="Actual y")
plt.xlabel("Waypoint")
plt.ylabel("Y position (m)")
plt.title("Desired vs Actual Y Position")
plt.legend()
plt.grid(True)
plt.show()

plt.figure()
plt.plot(waypoints, pz_desired, label="Desired z")
plt.plot(waypoints, pz_actual, label="Actual z")
plt.xlabel("Waypoint")
plt.ylabel("Z position (m)")
plt.title("Desired vs Actual Z Position")
plt.legend()
plt.grid(True)
plt.show()
```